

Aula 2

Grandes Modelos de Linguagem

Gabriel Lenz Balatka

Orientador: Rafael Stubs Parpinelli

Joinville, 15 de junho de 2026



UDESC
UNIVERSIDADE
DO ESTADO DE
SANTA CATARINA

LABICOM
Laboratório de Pesquisa
em Inteligência Computacional

Sumário

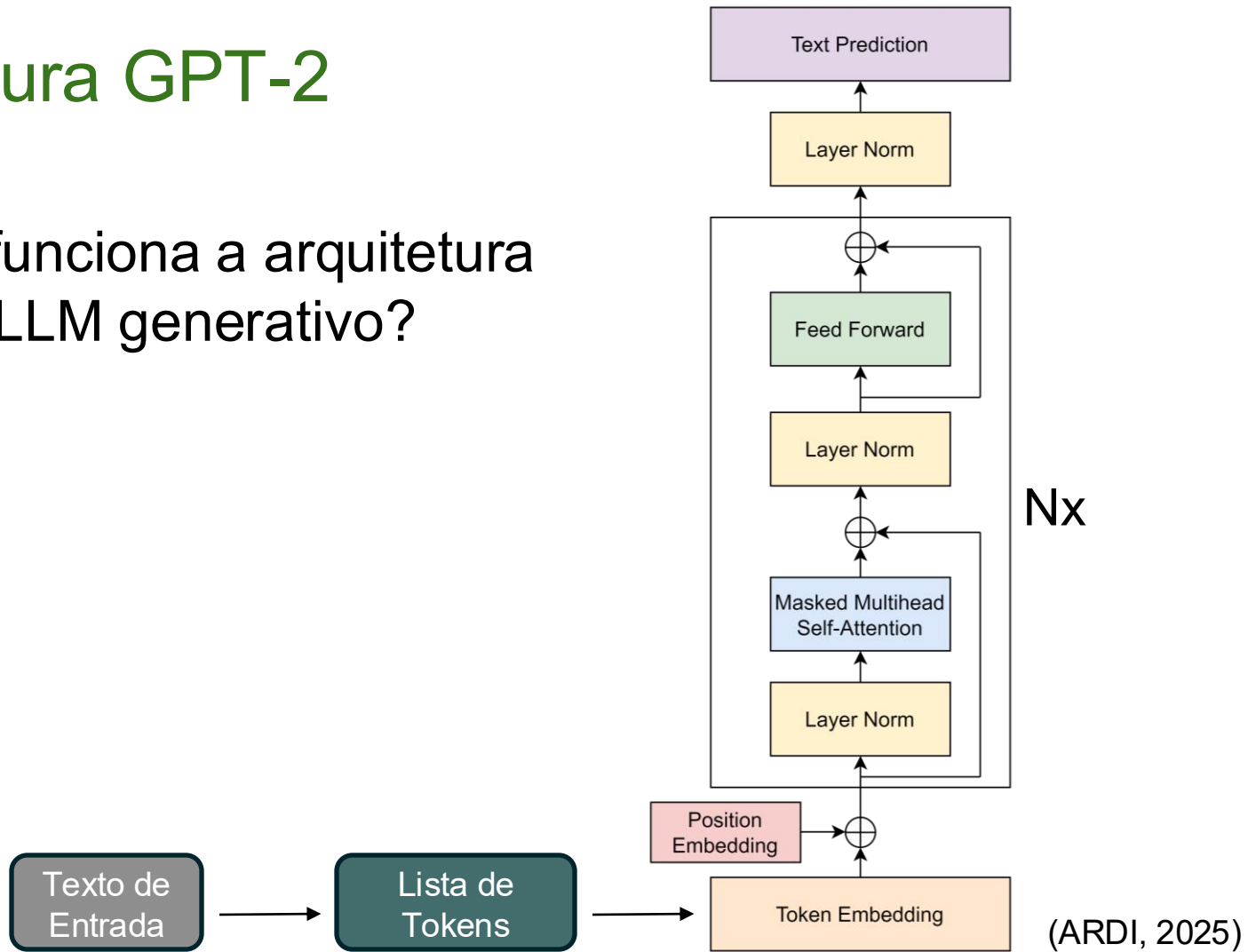
- Exercício da última aula
- Revisão da etapa de tokenização
- Continuação da construção de embeddings
- Estudo do processo de normalização

Exercício da Última Aula

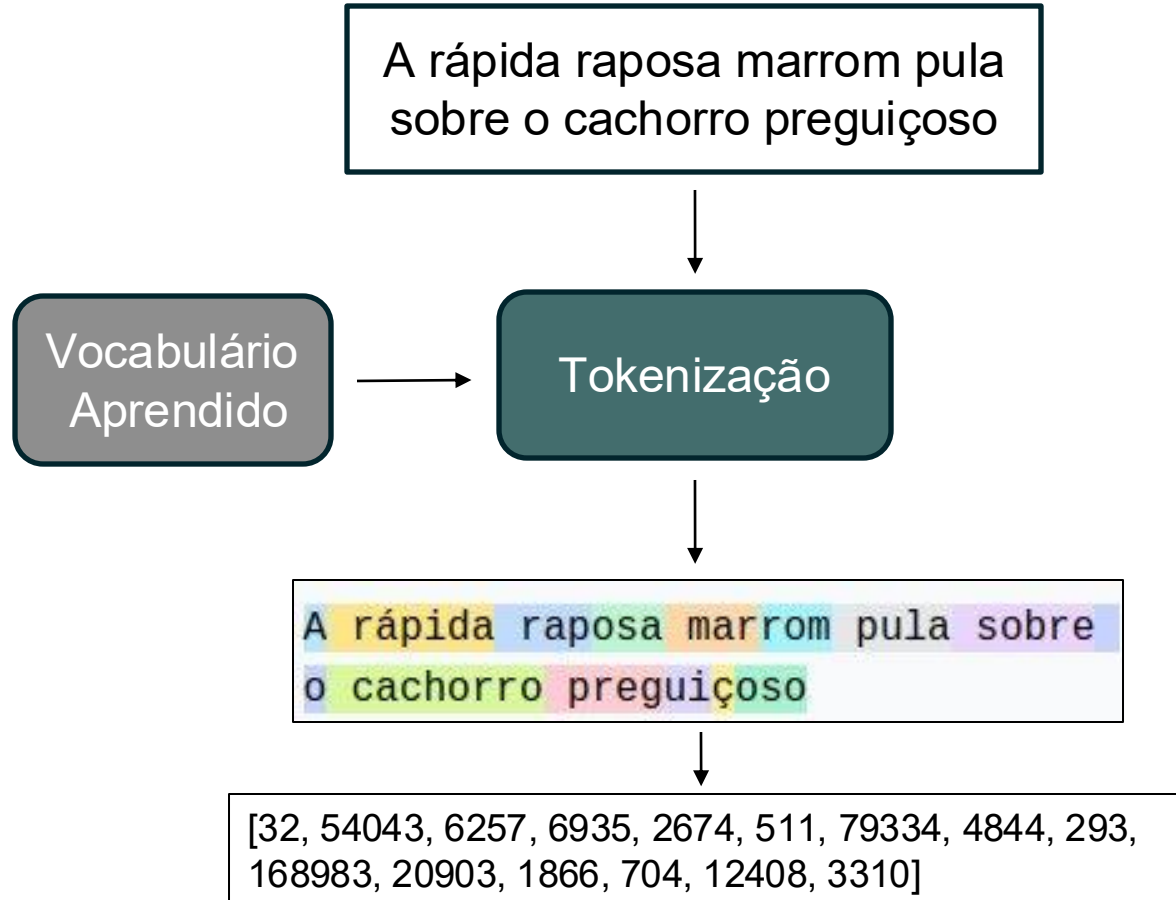
- Buscar um artigo que utilizou um modelo de linguagem baseado na arquitetura Transformer para a resolução de um problema de seu interesse
- Apresentar para a turma o nome do modelo utilizado, tipo (encoder-only, decoder-only ou encoder-decoder), quantidade de parâmetros e o problema solucionado
- Os resultados foram positivos?
- Quais as limitações encontradas?

Arquitetura GPT-2

- Como funciona a arquitetura de um LLM generativo?



Aplicação da Tokenização



Byte-Pair Encoding (BPE) Adaptado

Quantidade

Palavras

Vocabulário

2

· new

{·, e, n, r, s, t, w, ne, new, ·r}

2

·r e new

1

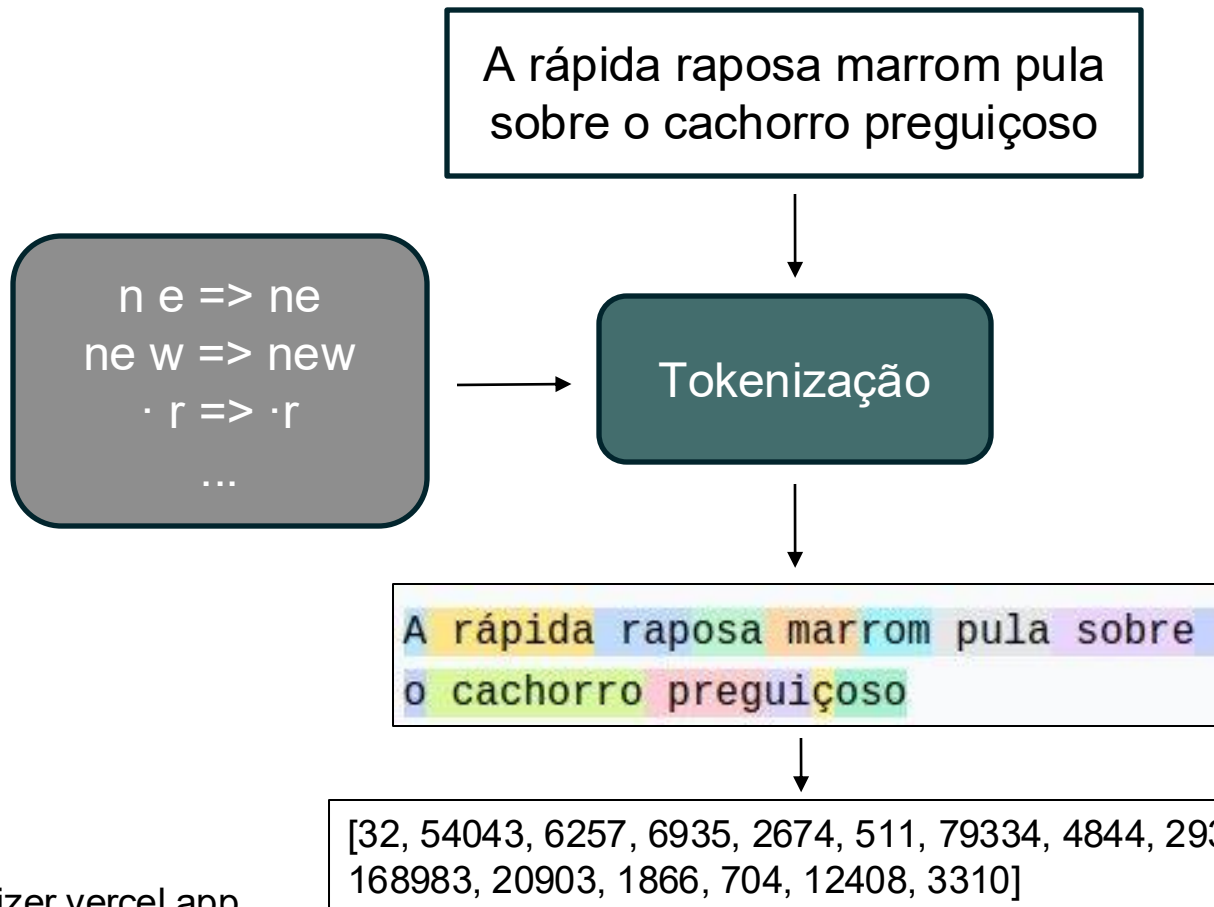
s e t

└→ Como utilizar este
vocabulário aprendido?

1

·r e s e t

Aplicação da Tokenização



Conversão de Tokens para Embeddings

- Os tokens são transformados em vetores numéricos para que o modelo consiga compreender a informação

1	Token ID		Dimension 0		Dimension 1		Dimension 2
2	-----						
3	0 (pad)		0.0		0.0		0.0
4	1 (hello)		0.23		-0.45		0.78
5	2 (world)		0.12		-0.33		0.89
6	3 (cat)		0.67		0.21		-0.12
7	...		...		...		...
8	50256		-0.34		0.56		0.23

(AKAN, 2025)

Contexto entre Embeddings

- Com isso, os embeddings contêm o significado dos tokens e o modelo deverá aprender a relação entre eles (contexto)
- O grande diferencial da arquitetura Transformer é o processamento em paralelo
- Mas quais problemas o processamento em paralelo traz?

Contexto entre Embeddings

- Qual a diferença entre as frases abaixo?
 - O cachorro mordeu o homem
 - O homem mordeu o cachorro

Ordem entre Embeddings

- Qual a diferença entre as frases abaixo?
 - O cachorro mordeu o homem
 - O homem mordeu o cachorro
- As frases contêm o mesmo conjunto de palavras
- Mas a ordem entre as palavras é diferente e assim o sentido também é alterado
- Como passar essa informação para o modelo?

Ordem entre Embeddings com Posição Absoluta

- Primeira estratégia: Considerar a posição absoluta de cada token
- Como isso pode ser feito?

Token embedding matrix

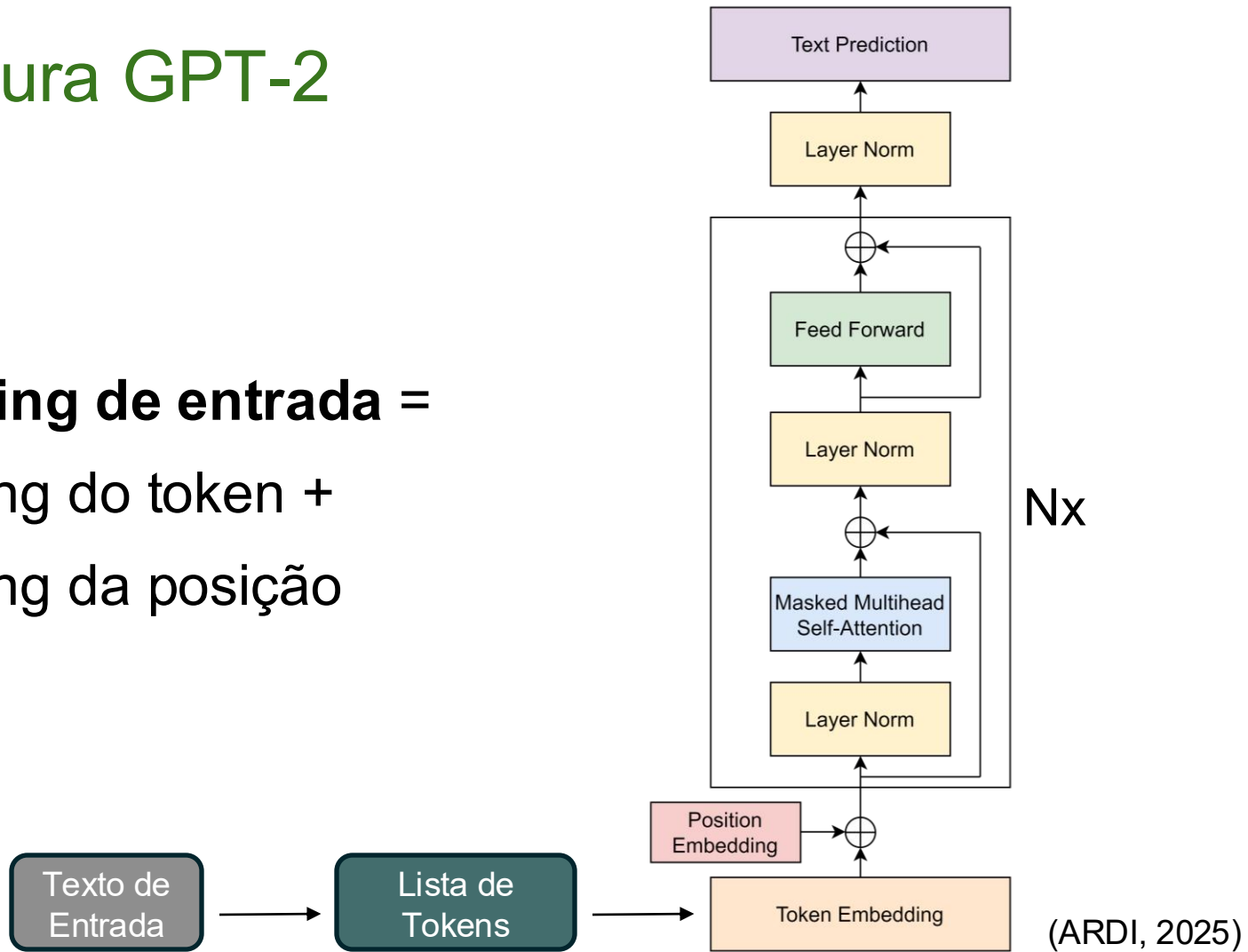
Token	dim_1	dim_2
O	0.4	-0.8
gato	-0.7	0.6
sentou	0.9	-0.1
no	-0.2	0.7
tapete	0.1	-0.5

- Frase de entrada:

O gato sentou no tapete

Arquitetura GPT-2

Embedding de entrada =
embedding do token +
embedding da posição



Ordem entre Embeddings com Posição Absoluta

Token embedding matrix

Token	dim_1	dim_2
O	0.4	-0.8
gato	-0.7	0.6
sentou	0.9	-0.1
no	-0.2	0.7
tapete	0.1	-0.5

Position embedding matrix

Pos	dim_1	dim_2
01	0.3	0.2
02	-0.9	-0.6
03	0.0	1.0
04	0.8	-0.3
05	-0.4	0.5

Input embedding matrix

Token	dim_1	dim_2
O		
gato		
sentou		
no		
tapete		

- Como calcular a *input embedding matrix*?

Ordem entre Embeddings com Posição Absoluta

Token embedding matrix

Token	dim_1	dim_2
O	0.4	-0.8
gato	-0.7	0.6
sentou	0.9	-0.1
no	-0.2	0.7
tapete	0.1	-0.5

Position embedding matrix

Pos	dim_1	dim_2
01	0.3	0.2
02	-0.9	-0.6
03	0.0	1.0
04	0.8	-0.3
05	-0.4	0.5

Input embedding matrix

Token	dim_1	dim_2
O	0.7	-0.6
gato	-1.6	0.0
sentou	0.9	0.9
no	0.6	0.4
tapete	-0.3	0.0

Ordem entre Embeddings com Posição Absoluta

- A matriz de posição pode ser fixa ou aprendida
- **Matriz Fixa:** Utilização de senos e cossenos de diferentes frequências para a criação de padrões únicos em cada posição
- **Matriz Aprendida:** Inicialização aleatória e aprendida durante o treinamento

Quais as vantagens e desvantagens de cada estratégia?

Position embedding matrix

Pos	dim_1	dim_2
01	0.3	0.2
02	-0.9	-0.6
03	0.0	1.0
04	0.8	-0.3
05	-0.4	0.5

Ordem entre Embeddings com Posição Absoluta

- A arquitetura original Transformer utilizou uma matriz fixa para a codificação das posições

$$\begin{aligned} PE_{(pos, 2i)} &= \sin(pos/10000^{2i/d_{model}}) \\ PE_{(pos, 2i+1)} &= \cos(pos/10000^{2i/d_{model}}) \end{aligned}$$

(VASWANI, 2017)

- pos é a posição do token
- i é a posição da dimensão
- d_{model} é a dimensão dos embeddings

Ordem entre Embeddings com Posição Absoluta

- Os autores hipotetizaram que esta codificação permitiria ao modelo aprender facilmente a prestar atenção com base em posições relativas
 - 'k posições para frente' é sempre a mesma transformação linear, independente da posição inicial
- Testes também foram feitos com o treinamento da matriz de posições e resultados muito similares foram encontrados

Ordem entre Embeddings com Posição Absoluta

- Além disso, a matriz fixa pode permitir ao modelo lidar com contextos maiores do que os vistos no treinamento
- Mesmo assim, os modelos GPT-2 e GPT-3 utilizam uma matriz aprendida durante o treinamento

Outras formas de considerar a posição dos embeddings

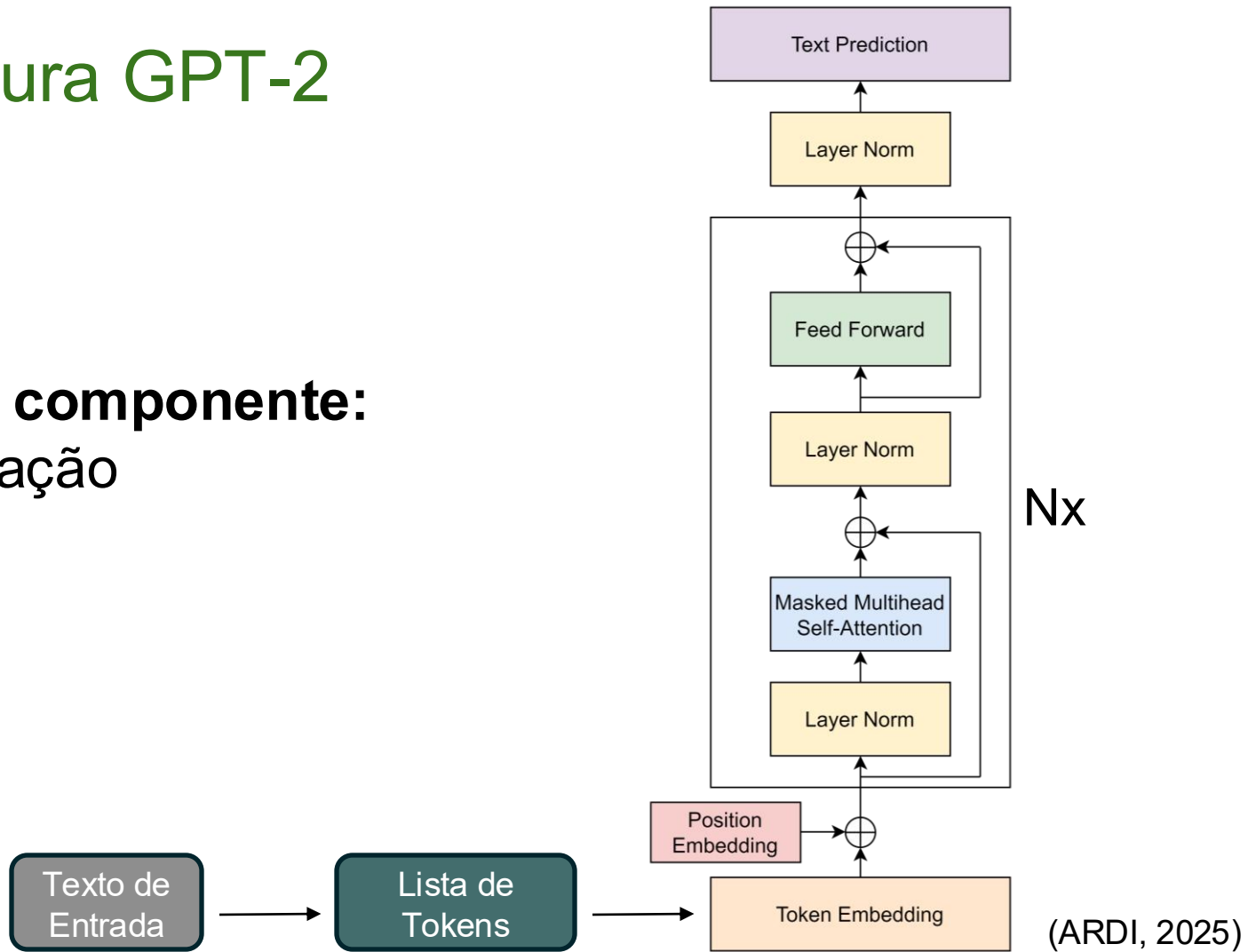
- Embeddings posicionais relativos
 - Utilização da distância relativa entre os tokens
 - Relação entre os tokens é influenciada pela distância
- Embeddings posicionais rotativos
 - Leva em consideração a posição relativa durante o cálculo da relação entre os tokens de forma eficiente

Revisão das estratégias de embeddings posicionais

- Posição absoluta de cada token
 - Fixa
 - Apreendida por treinamento
- Posição Relativa de cada token
 - Tradicional
 - Rotativa

Arquitetura GPT-2

Próximo componente:
Normalização



Normalização

- A normalização é aplicada duas vezes em cada camada do decoder
 - Antes da camada de atenção
 - Antes da rede feed forward
- Além disso, é aplicada antes da predição de texto
- Quais as vantagens da aplicação da normalização?

Normalização

- Quais as vantagens da aplicação da normalização?
- Melhoria da estabilidade do modelo durante o treinamento
 - Sem a normalização, funções de ativação podem gerar valores muito grandes ou muito pequenos e atrapalhar o treinamento

Normalização

- Para isso, primeiro começamos com a matriz de entrada
- E calculamos a média e a variância de cada embedding
Média ($E[x]$) = 0.05
Variância ($Var[x]$) = 0.42
- Em seguida, cada valor é subtraído pela média e dividido pelo desvio padrão

Input embedding matrix

Token	dim_1	dim_2
O	0.7	-0.6
gato	-1.6	0.0
sentou	0.9	0.9
no	0.6	0.4
tapete	-0.3	0.0

Normalização

- Primeira dimensão:
$$(0.7 - E[x]) / \sqrt{\text{Var}[x] + \epsilon} =$$
$$(0.7 - 0.05) / \sqrt{0.42 + \epsilon} =$$
$$1.00$$
- Segunda dimensão:
$$(-0.6 - E[x]) / \sqrt{\text{Var}[x] + \epsilon} =$$
$$(-0.6 - 0.05) / \sqrt{0.42 + \epsilon} =$$
$$-1.00$$
- Quais propriedades nós garantimos?

Token	dim_1	dim_2
O	0.7	-0.6
gato	-1.6	0.0
sentou	0.9	0.9
no	0.6	0.4
tapete	-0.3	0.0



Token	dim_1	dim_2
O	1.00	-1.00
gato	...	...
sentou	...	...
no	...	...
tapete	...	...

Normalização

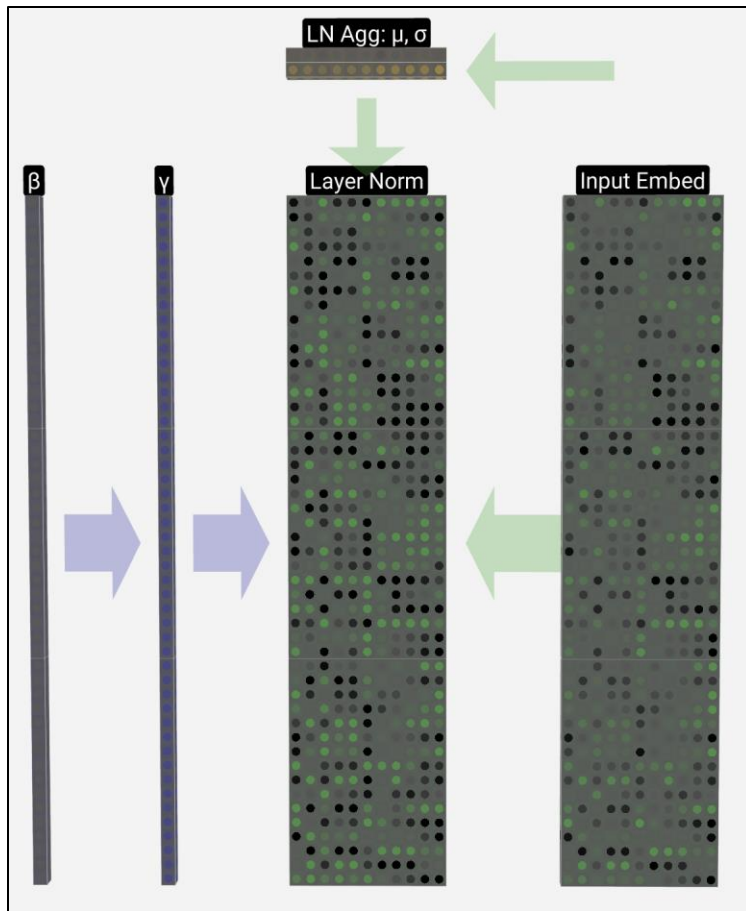
- Quais propriedades nós garantimos?
- Média igual a zero
- Desvio padrão igual a um
- Por fim, nós multiplicamos cada valor da dimensão por um peso (γ) e somamos um bias (β)
- Ambas as constantes são aprendidas durante o treinamento

Token	dim_1	dim_2
O	0.7	-0.6
gato	-1.6	0.0
sentou	0.9	0.9
no	0.6	0.4
tapete	-0.3	0.0



Token	dim_1	dim_2
O	1.00	-1.00
gato	...	...
sentou	...	...
no	...	...
tapete	...	...

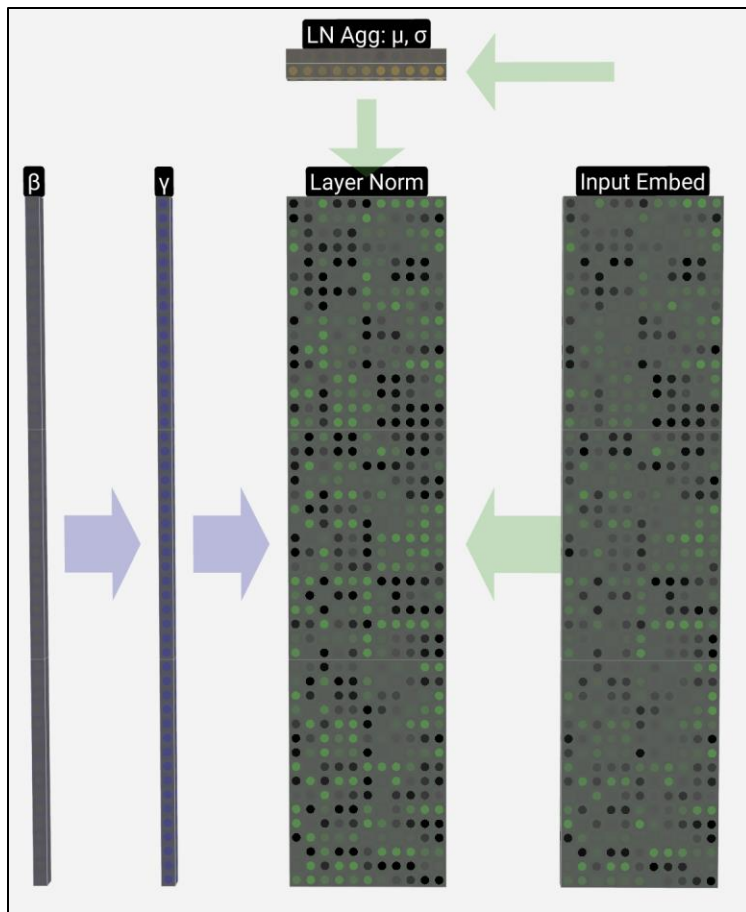
Visualização da Normalização



- Por que multiplicar cada dimensão por um peso (γ) e somar um bias (β)?

(BYCROFT, 2023)

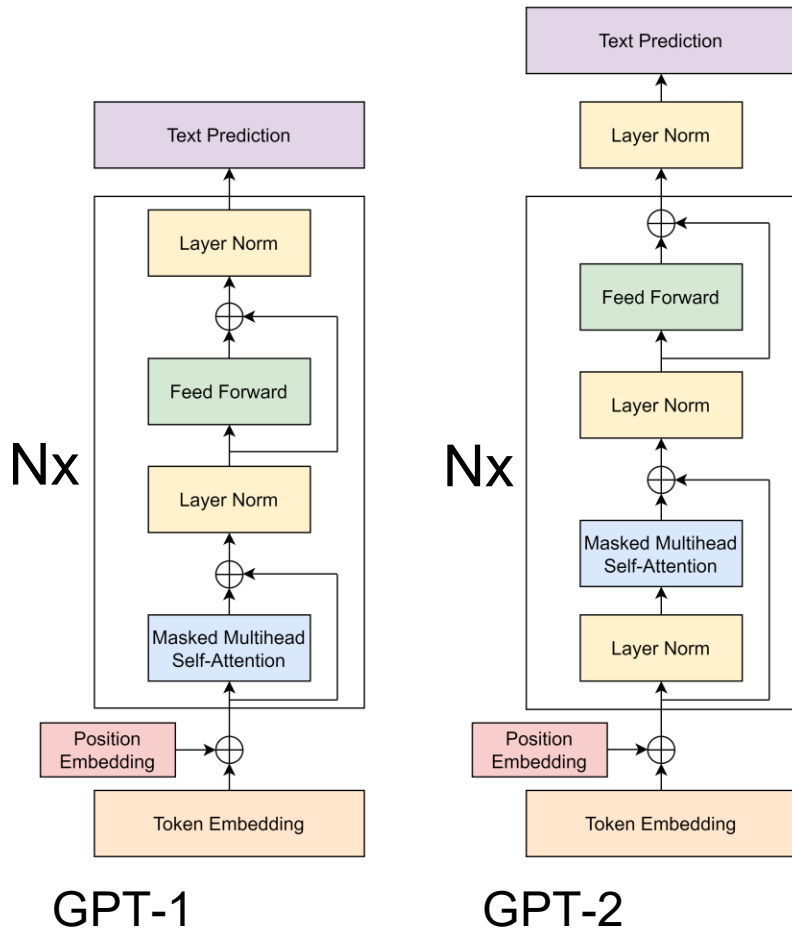
Visualização da Normalização



- Por que multiplicar cada dimensão por um peso (γ) e somar um bias (β)?
- A multiplicação por peso(γ) e soma de um bias (β) aumentam a flexibilidade do modelo
- Sendo possível, até mesmo desfazer a normalização

(BYCROFT, 2023)

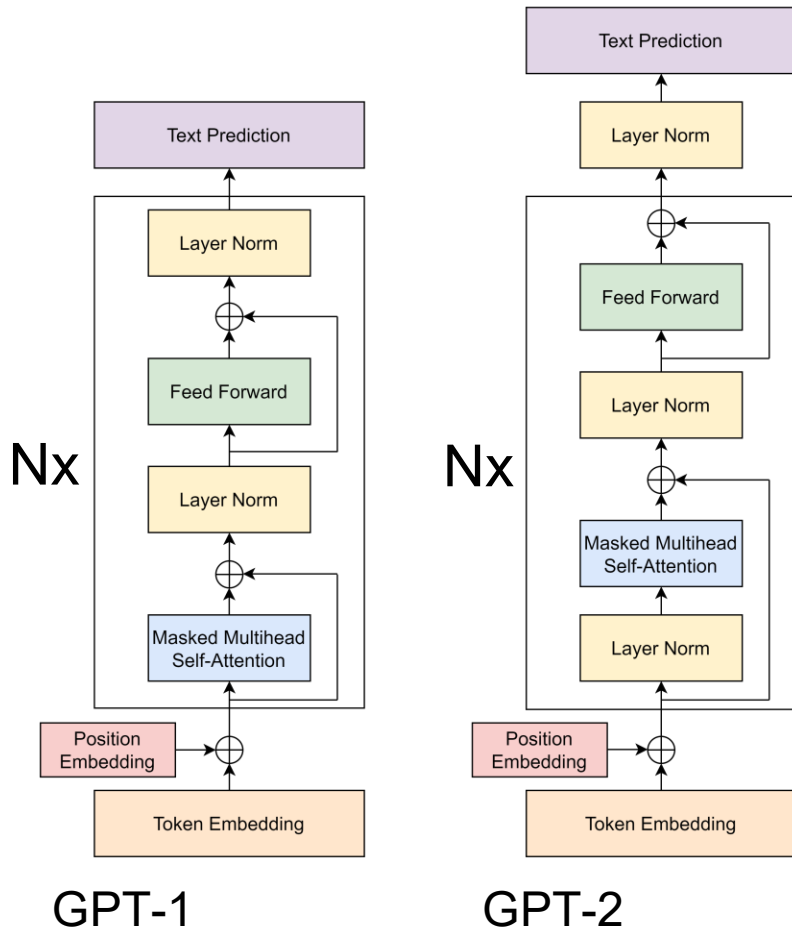
Visualização da Normalização



- Qual a diferença entre as duas arquiteturas?

(ARDI, 2025)

Visualização da Normalização



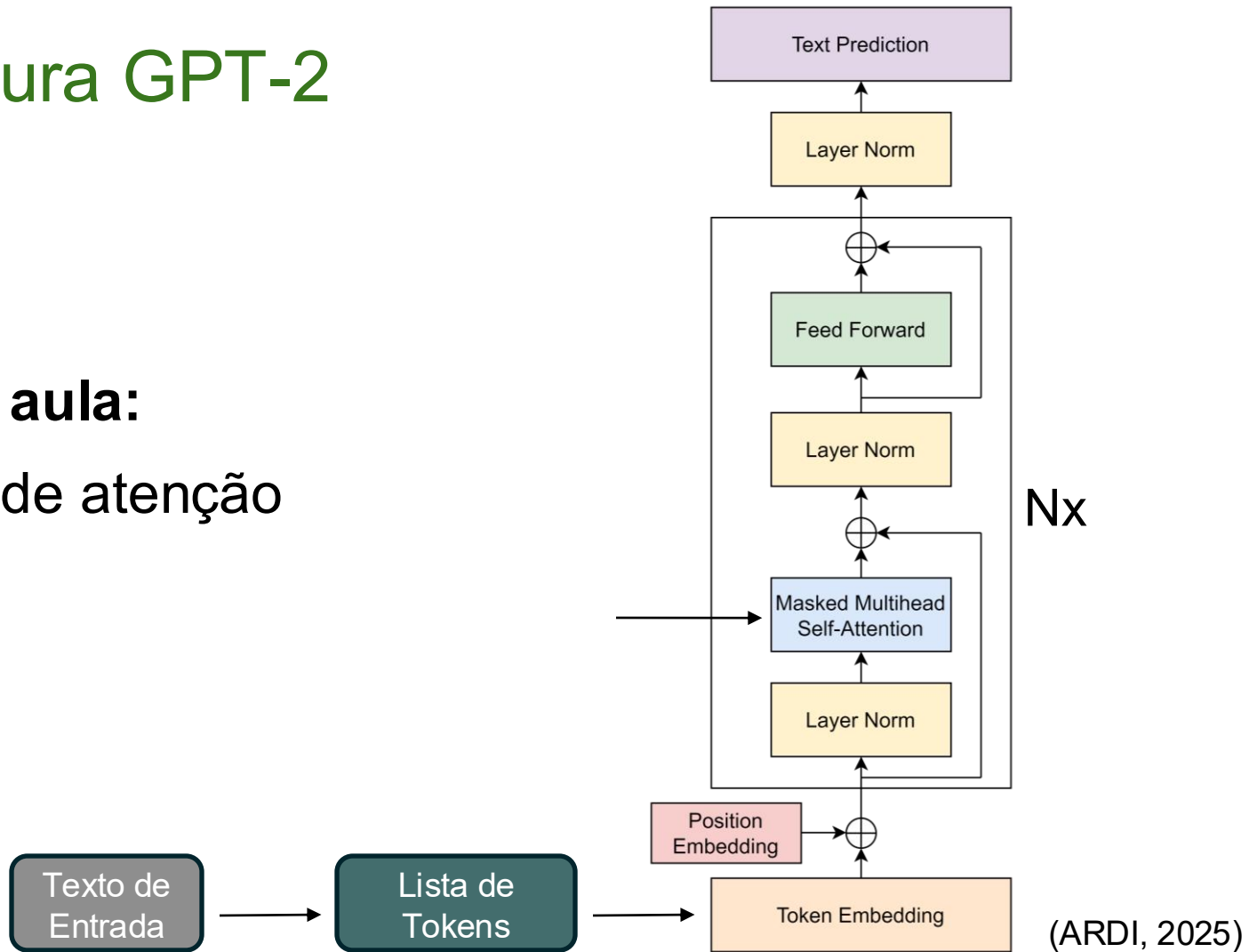
- Qual a diferença entre as duas arquiteturas?
- O GPT-1 utiliza post-normalization
- O GPT-2 utiliza pre-normalization

(ARDI, 2025)

Arquitetura GPT-2

Próxima aula:

Camada de atenção



Referências

- JURAFSKY, Daniel; MARTIN, James H. **Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models.** 3. ed. Stanford: Stanford University, 2026.
- VASWANI, Ashish et al. **Attention is all you need.** Advances in neural information processing systems, v. 30, 2017.
- FERRER, Josep. **How Transformers Work: a detailed exploration of transformer architecture.** DataCamp, 2026.
- BENTOML. **How does LLM inference work?** LLM Inference Handbook, 2025.

Referências

- MINAEE, Shervin et al. **Large language models: A survey**. arXiv preprint arXiv:2402.06196, 2024.
- NOBLE, Joshua. **Encoder-decoder model**. IBM, 2025.
- AKAN, Oz. **Words, Tokens and Embeddings**. Luminary Blog, 2025.
- ARDI, Muhammad. **Meet GPT, The Decoder-Only Transformer**. Towards Data Science, 2025.
- ARTICSLEDGE. **What Is Positional Encoding? How AI Models Know Word Order (2026)**. Articsledge, 2026.

Referências

- GHASHAMI, Mina. **Understanding positional embeddings in transformers**: from absolute to rotary. Towards Data Science, 2024.
- BYCROFT, Brendan. **LLM Visualization**. Disponível em: bbycroft.net/llm.

Aula 2

Grandes Modelos de Linguagem

Gabriel Lenz Balatka

Orientador: Rafael Stubs Parpinelli

Joinville, 15 de junho de 2026



UDESC
UNIVERSIDADE
DO ESTADO DE
SANTA CATARINA

LABICOM
Laboratório de Pesquisa
em Inteligência Computacional